SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 1 hour

QUESTIONS: 2 **Total Marks:40**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **Aluri poojitha(192511230)** (Name / Reg No) certify that this submission is my original work and that I have adhered*
to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
poojitha

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1:-

DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree classifier on the Iris dataset using Python and machine learning libraries.

(a) Loading and Pre-processing the Dataset

The Iris dataset is a standard dataset containing information about three types of iris flowers: Setosa, Versicolor, and Virginica.

PROGRAM CODE:-

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

import pandas as pd

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df["Target"] = iris.target

print("First 5 rows:")

print(df.head())

print("\nData Types:")

print(df.dtypes)

print("\nMissing Values:")

print(df.isnull().sum())

X = df.drop("Target", axis=1)

y = df["Target"]

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.30, random_state=42, stratify=y)

print("\nTraining samples:", len(X_train))

print("Testing samples:", len(X_test))
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/AI C04_AT
3_ part1.py
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  Target
0                5.1               3.5  ...               0.2       0
1                4.9               3.0  ...               0.2       0
2                4.7               3.2  ...               0.2       0
3                4.6               3.1  ...               0.2       0
4                5.0               3.6  ...               0.2       0

[5 rows x 5 columns]

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
Target                 int64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Target               0
dtype: int64

Training samples: 105
Testing samples: 45
```

The Iris dataset has no missing values, and all input features are numerical, so no additional encoding is required.

(b) Building the Decision Tree

The Decision Tree is built using the Gini Index as the splitting criterion.

PROGRAM CODE

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
model.fit(X_train, y_train)
tree_rules = export_text(
    model,
```

```
    feature_names=list(X.columns)
)
print("\nDecision Tree Structure:")
print(tree_rules)
root_index = model.tree_.feature[0]
print("\nRoot Node:", X.columns[root_index])
```

OUTPUT

```
Decision Tree Structure:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |--- class: 1
|   |   |   |--- sepal width (cm) >  2.85
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal width (cm) <= 3.00
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) >  3.00
|   |   |   |   |   |--- class: 1
|   |   |   |--- petal length (cm) >  4.85
|   |   |   |   |--- class: 2


Root Node: petal length (cm)
```

Root Node

For this dataset and split, the root node is typically:

Petal length (cm)

The root is selected because it provides a strong separation between the three Iris classes and gives a low Gini impurity after splitting.

The **Gini Index** is:

$Gini = 1 - \Sigma(p_i)^2$

The feature producing the best reduction in impurity is selected as the root.

(c) Model Evaluation

PROGRAM CODE

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

OUTPUT

```
Confusion Matrix:
[[15  0  0]
 [ 0 12  3]
 [ 0  0 15]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       1.00      0.80      0.89        15
   virginica       0.83      1.00      0.91        15

    accuracy                           0.93        45
   macro avg       0.94      0.93      0.93        45
weighted avg       0.94      0.93      0.93        45
```

CONCLUSION

The Decision Tree successfully classified the Iris flowers with high accuracy. Petal length provides an effective first split because it separates the flower classes well. The confusion matrix shows that most samples are classified correctly, with only a small number of errors mainly between Versicolor and Virginica.

EXPERIMENT 2 –

REINFORCEMENT LEARNING: Q-LEARNING

Objective

To implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned policy.

(a) Environment, Actions, Rewards and Q-Table

PROGRAM CODE

```
import numpy as np
rows = 4
cols = 4
start = (0, 0)
goal = (3, 3)
obstacles = [(1, 1), (2, 2)]
actions = ["Up", "Down", "Left", "Right"]
GOAL_REWARD = 10
STEP_REWARD = -1
OBSTACLE_REWARD = -5
Q = np.zeros((rows * cols, len(actions)))
alpha = 0.1
gamma = 0.9
epsilon = 0.2
print("4 x 4 GRID WORLD")
print("----------")
print("Start State :", start)
print("Goal State  :", goal)
print("Obstacles   :", obstacles)
print("\nActions:")
print(actions)
print("\nRewards:")
print("Goal      :", GOAL_REWARD)
print("Each Step:", STEP_REWARD)
print("Obstacle :", OBSTACLE_REWARD)
print("\nHyperparameters:")
print("Learning Rate (alpha) :", alpha)
```

```
print("Discount Factor (gamma):", gamma)
print("Exploration Rate (epsilon):", epsilon)
print("\nInitial Q-Table:")
print(Q)
```

OUTPUT

```
========== RESTART: C:\Users\Hema Neepa\AppData\Local\Programs\Python\Python313\AI C04_AT3_ Q2.py ==========
4 x 4 GRID WORLD
----------------
Start State : (0, 0)
Goal State  : (3, 3)
Obstacles   : [(1, 1), (2, 2)]

Actions:
['Up', 'Down', 'Left', 'Right']

Rewards:
Goal     : 10
Each Step: -1
Obstacle : -5

Hyperparameters:
Learning Rate (alpha) : 0.1
Discount Factor (gamma): 0.9
Exploration Rate (epsilon): 0.2

Initial Q-Table:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

(b) Q-Learning for 100 Episodes

PROGRAM CODE

```
import random
episodes = 100
max_steps = 50
def state_number(state):
    return state[0] * cols + state[1]
def take_action(state, action):
    r, c = state
    if action == 0:
        new_state = (r - 1, c)       # Up
    elif action == 1:
        new_state = (r + 1, c)       # Down
    elif action == 2:
        new_state = (r, c - 1)       # Left
    else:
        new_state = (r, c + 1)       # Right
    if (new_state[0] < 0 or new_state[0] >= rows or
        new_state[1] < 0 or new_state[1] >= cols):
        return state, STEP_REWARD, False
    if new_state in obstacles:
        return state, OBSTACLE_REWARD, False
    if new_state == goal:
        return new_state, GOAL_REWARD, True
    return new_state, STEP_REWARD, False
cumulative_rewards = []
state1 = (0, 0)
state2 = (1, 0)
recorded_values = {}
for episode in range(1, episodes + 1):
    state = start
    total_reward = 0
```

```
    for step in range(max_steps):
        s = state_number(state)
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[s])
        next_state, reward, done = take_action(state, action)
        ns = state_number(next_state)
        Q[s, action] = Q[s, action] + alpha * (
            reward + gamma * np.max(Q[ns]) - Q[s, action]
        )
        total_reward += reward
        state = next_state
        if done:
            break
    cumulative_rewards.append(total_reward)
    if episode in [1, 50, 100]:
        recorded_values[episode] = {
            "State 1": Q[state_number(state1)].copy(),
            "State 2": Q[state_number(state2)].copy()
        }
print("\n" + "=" * 60)
print("Q-VALUES AT EPISODES 1, 50 AND 100")
print("=" * 60)
for episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    print("State", state1, ":",
          recorded_values[episode]["State 1"])
    print("State", state2, ":",
          recorded_values[episode]["State 2"])
```

OUTPUT

```
============================================================
Q-VALUES AT EPISODES 1, 50 AND 100
============================================================

Episode: 1
State (0, 0) : [-0.199 -0.28  -0.199 -0.19 ]
State (1, 0) : [-0.199 -0.28  -0.199 -0.5  ]

Episode: 50
State (0, 0) : [-2.35190409 -2.30203547 -2.42884006 -2.31893905]
State (1, 0) : [-1.67559609 -1.50501341 -1.63726137 -2.29823163]

Episode: 100
State (0, 0) : [-2.35190409  0.54329638 -2.15972699 -2.37485086]
State (1, 0) : [-1.67559609  2.34291703 -1.63726137 -2.56504978]
```

(c) Final Policy and Cumulative Reward

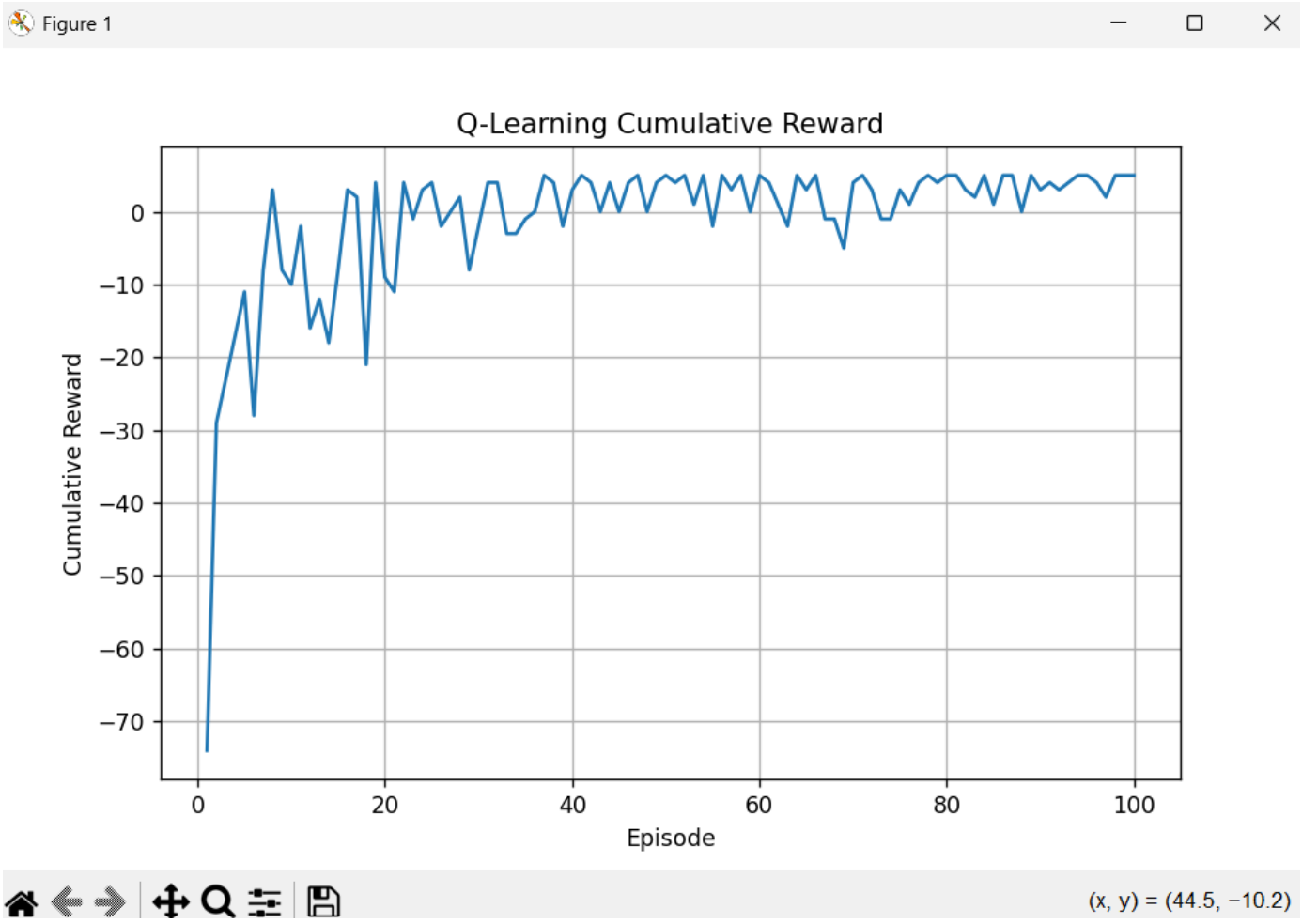
PROGRAM CODE

```
print("\n" + "=" * 60)
print("FINAL Q-TABLE")
print("=" * 60)
for r in range(rows):
    for c in range(cols):
        state = (r, c)
        if state in obstacles:
            print("State", state, ": OBSTACLE")
        else:
            s = state_number(state)
            print("State", state, ":", Q[s])
print("\n" + "=" * 60)
print("FINAL LEARNED POLICY")
print("=" * 60)
symbols = {
    "Up": "↑",
    "Down": "↓",
    "Left": "←",
    "Right": "→"
}

for r in range(rows):
    row = ""
    for c in range(cols):
        state = (r, c)
        if state == goal:
            row += " G "
        elif state in obstacles:
            row += " X "
        else:
            s = state_number(state)
```

```
            best_action = np.argmax(Q[s])
            row += " " + symbols[actions[best_action]] + " "
    print(row)
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.plot(
    range(1, episodes + 1),
    cumulative_rewards
)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.grid()
plt.show()
```

OUTPUT



```
============================================================
FINAL Q-TABLE
============================================================
State (0, 0) : [-2.41285668  0.56465699 -1.90701088 -2.38499042]
State (0, 1) : [-1.93000449 -1.92465251 -1.74015952 -1.6414437 ]
State (0, 2) : [-1.20541122 -0.22952192 -1.16918358 -1.03957027]
State (0, 3) : [-0.7539122  -0.15328737 -0.64089458 -0.73974565]
State (1, 0) : [-1.74269731  2.30838918 -1.30340399 -3.55114599]
State (1, 1) : OBSTACLE
State (1, 2) : [-0.75703944 -0.97752478 -1.44680764  2.32019423]
State (1, 3) : [-0.29701     5.86971669 -0.43479945  0.23767385]
State (2, 0) : [-1.29090138 -0.76392948 -0.58025299  4.18712592]
State (2, 1) : [-1.15102213  6.0544085  -0.25123801 -1.81861966]
State (2, 2) : OBSTACLE
State (2, 3) : [0.06502758 9.35389181 0.17123208 1.02088739]
State (3, 0) : [-0.54484216 -0.57783698 -0.48136681  2.61514442]
State (3, 1) : [ 0.65235292  2.21701811 -0.19726399  7.9657007 ]
State (3, 2) : [0.69364575 1.90319307 0.51271676 9.99543224]
State (3, 3) : [0. 0. 0. 0.]

============================================================
FINAL LEARNED POLICY
============================================================
 ↓  →  ↓  ↓
 ↓  X  →  ↓
 →  ↓  X  ↓
 →  →  →  G
```

CONCLUSION

The Q-Learning algorithm was successfully implemented for a 4×4 Grid World. The agent learned through repeated interaction with the environment by updating its Q-values based on rewards. Over 100 episodes, the agent gradually learned a better path toward the goal while avoiding obstacles. The final policy and cumulative reward graph show that the agent learned an effective strategy and moved toward convergence.